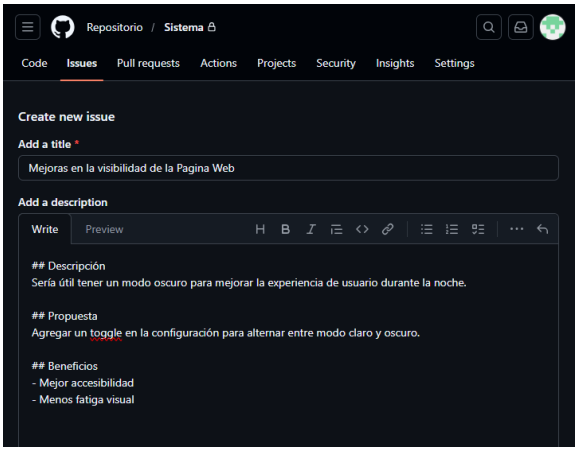
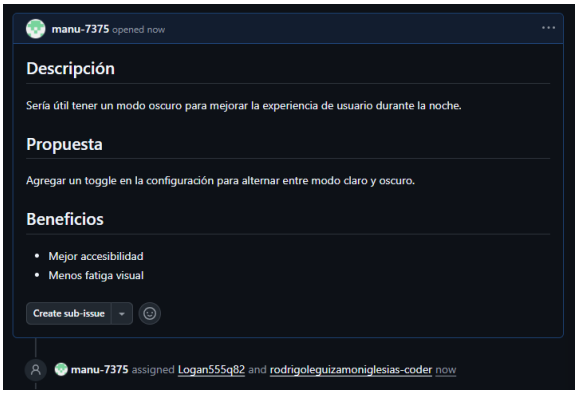
¿Qué es GitHub Issues?

GitHub Issues es una herramienta de GitHub que se usa para realizar el seguimiento de ideas, comentarios, tareas o errores.
Ayuda a mantener una organización apta en el espacio de trabajo, permitiendo realizar el seguimiento correspondiente de la o las distintas áreas de trabajo correspondientes, y la idea principal es prevenir y solucionar posibles errores o amenazas que el trabajo pueda tener de una forma mas organizada y detallada.
Varios usos que se le pueden dar son Reportar errores, Proponer mejoras, Discutir ideas o cambios, etc, y también pueden relacionarse con commits y pull requests.



En este caso se muestra un escenario ficticio de como podría usarse la función de Issues en un Repositorio con Colaboradores, aunque no llega a verse, también se brindan las opciones para añadir etiquetas, asignar personas, adjuntar archivos o proyectos, entre otros.



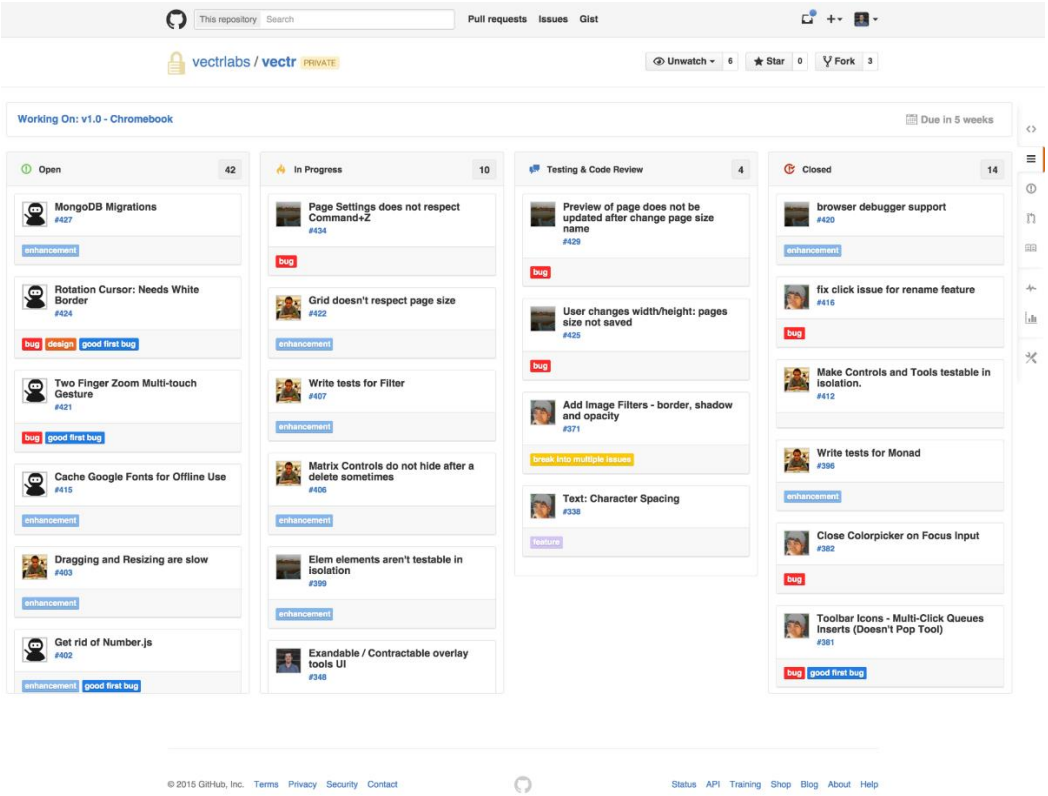
Los demás integrantes pueden participar brindando sus contribuciones, y cuando las expectativas o el principal quede resuelto, el Issue puede marcarse como 'Resuelto'

Las capturas previas son casos Ilustrativos, cuyo contenido fue creado por ChatGPT*
Fuente del contenido: https://docs.github.com/es/issues/tracking-your-work-with-issues/learning-about-issues/about-issues

¿Qué son los proyectos "KanBan" en GitHub?

Los proyectos Kanban son aquellos que usan un sistema de organización por tablas, la cuál se basa en los datos que hay en GitHub y en donde los proyectos realizan seguimiento de incidencias o solicitudes de cambios e ideas.

Es una herramienta bastante amplia, tiene muchas herramientas de personalización, permitiendo crear, visualizar, editar, ordenar y agrupar los datos de la manera que sean mas convenientes para el proyecto en cuestión y sus diferentes etapas.



Fuente de la Imagen: https://github.com/vectrlabs/GitHub-Kanban

En la imagen se puede llegar a notar un proyecto que usa esta forma de trabajo mencionada previamente, se llega a notar la organización por columnas que muestra las distintas etapas del proyecto y por las distintas versiones por las cuales se pasó.

Se llega a ver que en cada entrega hay etiquetas y nombres correspondientes al estado de la entrega y la información de la misma, esto más el sistema de columnas ayuda a que las distintas etapas y versiones del proyecto puedan diferenciarse más fácilmente, manteniendo una organización bastante limpia.

Fuente de la información: https://docs.github.com/es/issues/planning-and-tracking-with-projects

¿Como acceder Remotamente mediante Git Bash a GitHub (Incluir Testeo + Funcionalidad)

Primeramente, para realizar este procedimiento, vamos a precisar Descargar e Instalar el programa “Git Bash”, cuyo procedimiento varia dependiendo del Sistema Operativo donde se quiera instalar.

También será necesario iniciar sesión previamente en GitHub usando un navegador web o ya contar con una sesión abierta en el mismo.

Instalador de Git Bash (Todos los Sistemas Operativos de Escritorio): https://git-scm.com/install/

1- Abrir Git Bash y crear una llave SSH:
(El SSH (O Shell Seguro) es un protocolo cliente-servidor para conectarse y autenticarse a un servidor remoto)

ssh-keygen -t ed25519 -C <correo>

2- Presionar ENTER para guardar los archivos en la ubicación predeterminada, si ya existe una, sobrescríbela.
3- Iniciamos el agente SSH con la siguiente linea: eval "$(ssh-agent -s)"
4- Añadimos la clave: ssh-add ~/.ssh/id_ed25519
5- Copiamos la clave publica: cat ~/.ssh/id_ed25519.pub
6- Abrimos el navegador web y vamos a la pagina home de GitHub.
7- Una vez en la nuestra pagina Home, presionamos nuestro perfil y vamos a: Settings >> SSH and GPG keys >> New SSH key y pega la clave.
(Asegurate de establecer el tipo de key como "Authentication Key" y de ingresar un Titulo correspondiente a la llave)
8- Volve a Git Bash y ejecuta la siguiente sentencia: ssh -T git@github.com

(Debería saltarte un mensaje similar al siguiente: "`Hi <user>! You've successfully authenticated.`")

Fuente: https://www.freecodecamp.org/espanol/news/como-funciona-internamente-la-autenticacion-ssh-con-github/#:~:text=El%20SSH%20(Shell%20Seguro)%20es,autenticarse%20a%20un%20servidor%20remoto.

Actividad de testeo

Para testear la conectividad entre Git Bash y GitHub, vamos a ejecutar una serie de comandos en la terminal para verificar que la conexión se realizo correctamente.

Crear y subir un repositorio local a la nube (incluyendo COMMIT)

1- Abrimos Git Bash y usando el comando 'cd' viajamos hasta donde queremos nuestro repositorio localmente

```
Manu@T140J2312005569 MINGW64 ~
$ cd Repositorios/Prueba/

Manu@T140J2312005569 MINGW64 ~/Repositorios/Prueba
$
```

(si la carpeta no existe, podes usar el comando 'mkdir <nombre>' para crear una nueva carpeta donde estes)

2- Una vez situados en nuestra "carpeta repositorio" ejecutamos la sentencia 'git init' para especificar que esta carpeta será nuestro nuevo repositorio (se creara un archivo .git oculto dentro de la misma)

```
Manu@T140J2312005569 MINGW64 ~/Repositorios/Prueba
$ git init
Initialized empty Git repository in C:/Users/Manu/Repositorios/Prueba/.git/

Manu@T140J2312005569 MINGW64 ~/Repositorios/Prueba (master)
$
```

2.5- Creamos un archivo 'Documento.txt' para añadir información al repositorio y le ponemos información.

3- Para subir nuestro repositorio local a la nube, tenemos que hacer los siguientes pasos:

a. Ubicarnos en nuestra "carpeta repositorio" y ejecutar la sentencia 'git add .' (esto prepara los archivos para el COMMIT)

```
Manu@T140J2312005569 MINGW64 ~/Repositorios/Prueba (master)
$ git add .
warning: in the working copy of 'documento.txt', LF will be replaced by CRLF the
 next time Git touches it
```

b. Antes de continuar, necesitamos brindarte a Git la identidad del autor, esto se logra usando cualquiera de los siguientes métodos:

git config --global user.email <correo>

git config --global user.name <usuario>

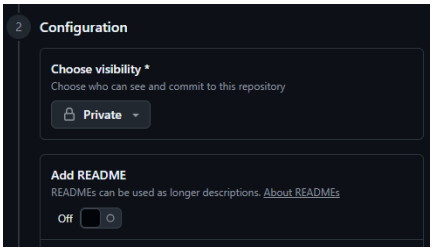
```
Manu@T140J2312005569 MINGW64 ~/Repositorios/Prueba (master)
$ git config --global user.email abraham2009mm@gmail.com
```

c. Realizar el commit correspondiente con la siguiente sentencia 'git commit -m "texto"'

```
Manu@T14OJ2312005569 MINGW64 ~/Repositorios/Prueba (master)
$ git commit -m "Commit de prueba"
[master (root-commit) d75271d] Commit de prueba
 1 file changed, 1 insertion(+)
 create mode 100644 documento.txt
```

d. Ahora abrimos al navegador y vamos a GitHub, donde crearemos un repositorio nuevo con el nombre que deseemos

(No añadas el archivo README)



e. Una vez creado el repositorio en la nube, copiamos su vinculo y volvemos a Git Bash, donde ejecutamos la siguiente sentencia
'git remote add origin <link al repositorio>'

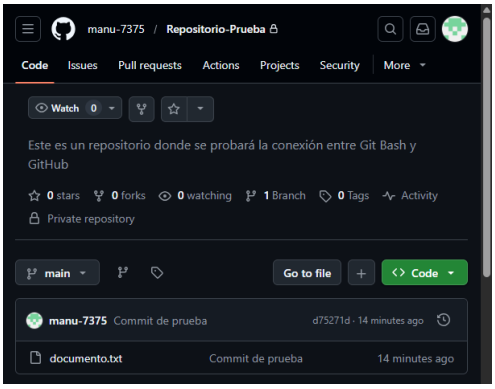
```
Manu@T14OJ2312005569 MINGW64 ~/Repositorios/Prueba (master)
$ git remote add origin https://github.com/manu-7375/Repositorio-Prueba.git
```

f. Como último paso, subimos el repositorio con las últimas 2 sentencias

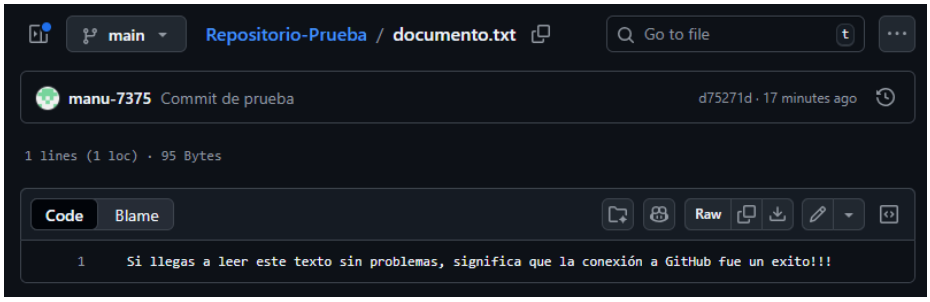
git branch -M <rama>
git push -u origin <rama>
(al hacer push existe la posibilidad de pedir acceso especial a GitHub, autorizar)

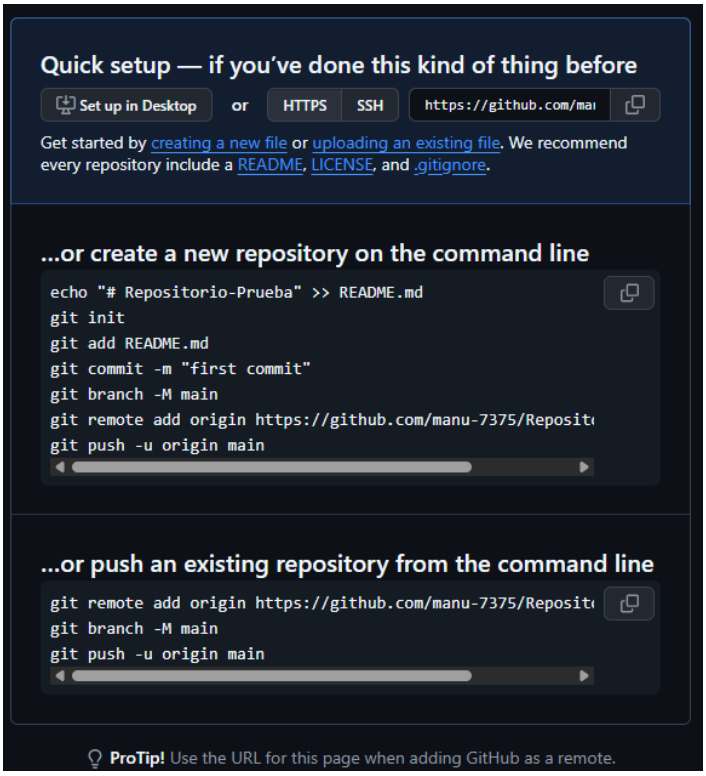
```
Manu@T14OJ2312005569 MINGW64 ~/Repositorios/Prueba (master)
$ git branch -M main

Manu@T14OJ2312005569 MINGW64 ~/Repositorios/Prueba (main)
$ git push -u origin main
info: please complete authentication in your browser...
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 2 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 300 bytes | 300.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/manu-7375/Repositorio-Prueba.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Despues de esto, para realizar otro Push solo necesitas preparar los archivos, realizar el commit correspondiente, y realizar el push.



Una vez hecho todo esto, vamos a nuestro navegador y abrimos GitHub, para verificar si nuestro Documento.txt se mando correctamente:





GitHub ya ofrece este tipo de ayuda de forma nativa, así que, si tenés problemas con las conexiones o con algún otro apartado, lo mejor será ver las guías oficiales para más detalles y más información.